IT5092 — NEUROSCIENCE & NEUROCOMPUTING

Assignment: Spiking Neural Network for MNIST Digit Classification

Assignment Sheet & Guidance

TYPE	TOTAL MARKS	SUBMISSION
Individual Assignment	**100 marks**	**One `.ipynb` file**

Learning Outcomes

By completing this assignment you will:

1. Implement a conductance-based Leaky Integrate-and-Fire (LIF) network in **Brian2**
2. Apply **Spike-Timing-Dependent Plasticity (STDP)** for unsupervised feature learning
3. Replicate the core architecture from **Diehl & Cook (2015)** and observe how it classifies MNIST digits without any labelled training signal
4. Connect biological learning mechanisms to the computational concepts from Weeks 3, 4, and 7

Reference: *Diehl, P.U. & Cook, M. (2015). Unsupervised learning of digit recognition using spike-timing-dependent plasticity. Frontiers in Computational Neuroscience, 9, 99.*

⚠ Submission Rules

- All cells must run **top-to-bottom without errors** before submission
- All TODO cells must be attempted
- Written analysis must be placed in **Markdown cells**
- Do **not** modify **provided** cells or change values in `config.py`

Mark Allocation

TODO	Section	Marks
TODO 1	Excitatory Neuron Group	**10**

TODO	Section	Marks
TODO 2	Inhibitory Neuron Group	**10**
TODO 3	Input→Excitatory STDP Synapses	**25**
TODO 4	Fixed Exc↔Inh Synapses	**5**
TODO 5	Training Loop	**15**
TODO 6	Test Phase	**15**
—	Visualisation (Plots)	**5**
TODO 7	Written Analysis (5 questions)	**15**
Total		**100**

How to Use This Sheet

Work through this document **alongside the notebook** `IT5092_Assignment_MNIST.ipynb`. Each section below corresponds directly to a section in the notebook. Read the explanation here first, then implement in the notebook.

> 💡 **TIP**
>
> The notebook already has all parameters imported from `config.py` and all utility functions imported from `utils/`. Your job is to fill in the TODO cells — the surrounding code is there to guide you.

Section 0: Brian2 Quick Introduction PROVIDED

The first working cell in the notebook contains a self-contained Brian2 example: a single LIF neuron driven by a constant current. Run it to verify your environment is set up

correctly. Study how Brian2 string equations, units, and `NeuronGroup` work — you will use the same patterns throughout.

Section 1: Load MNIST Data PROVIDED

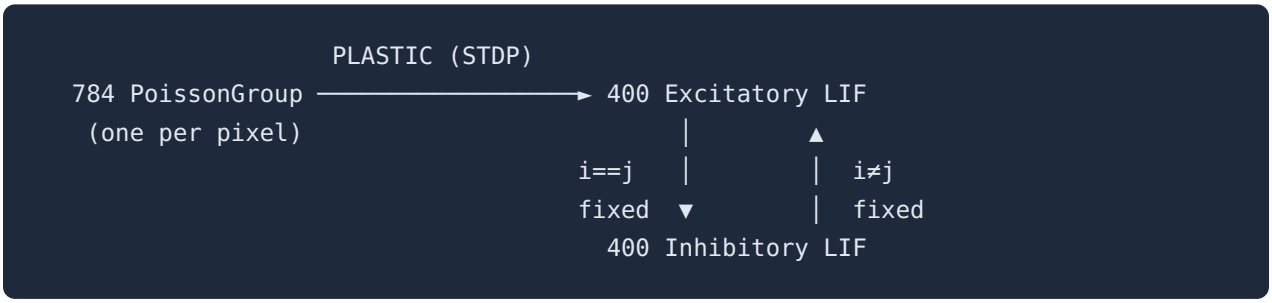
MNIST is a dataset of 70,000 handwritten digit images (28×28 pixels, greyscale). The notebook loads 1,000 training images and 50 test images from the `utils/mnist_loader.py` utility and visualises a sample alongside the rate-encoding scheme.

> 🧠 **BIOLOGICAL MOTIVATION: RATE CODING**
>
> Each pixel intensity (0–255) is converted to a Poisson spike rate. This mimics how sensory neurons encode stimulus intensity through their firing rate — a concept introduced in Week 3 (Poisson spike trains). A black pixel (0) produces a silent neuron; a white pixel (255) produces ~64 Hz of random spiking.

```
rate (Hz) = pixel / 8.0 × INPUT_INTENSITY_BASE    →    max rate ≈ 63.75 Hz
```

Section 2: Network Architecture

The network replicates Diehl & Cook (2015) exactly. Familiarise yourself with the architecture before implementing any code.



Component	Biological Motivation
Conductance-based LIF	Synaptic input opens ion channels; current depends on voltage relative to reversal potential (Week 4)
STDP on Input→Exc	Hebbian learning with causal timing: pre fires before post → strengthen (Week 7 + time direction)
One-to-one Exc→Inh	Each excitatory neuron activates its own dedicated inhibitory partner

Component	Biological Motivation
Lateral Inh→Exc (i≠j)	Winner-take-all: when one neuron fires, it suppresses all others via their inhibitory partner
Theta adaptation (θ)	Intrinsic plasticity: overactive neurons raise their own threshold, distributing load across the population

Section 3: Building the Network TODO

TODO 1 — Excitatory Neuron Group 10 marks

Build the main learning population: 400 LIF neurons with conductance-based synaptic input and an adaptive spiking threshold (θ).

> 🧠 **THE ADAPTIVE THRESHOLD?**
>
> If a neuron fires too frequently, θ grows, raising its effective threshold and making it harder to fire again. This is **intrinsic plasticity** — the neuron regulates its own excitability. The very slow decay ($\tau_\theta = 10^7$ ms ≈ 2.8 hours) means θ accumulates across the entire training session, permanently depressing overactive neurons and ensuring different neurons specialise on different digit patterns.

Steps to implement

a. **Verify the equation string.** The template in the notebook has all four ODEs. Check they match the description above.

b. **Create the NeuronGroup.** Fill in `threshold=`, `reset=`, and `refractory=REFRAC_E*ms` arguments.

c. **Initialise state variables.** Set `v = V_REST_E*mV`, `ge = 0`, `gi = 0`, `theta = THETA_INIT*mV`.

TODO 2 — Inhibitory Neuron Group

10 marks

Build the inhibitory population: 400 simpler LIF neurons — **no adaptive threshold**, faster time constant.

Steps to implement

a **Write `eqs_i`.** Three ODEs: `dv/dt`, `dge/dt`, `dgi/dt`. Use inhibitory parameters (`TAU_M_I`, `V_REST_I`, `E_INH_I`). No `dtheta/dt`.

b **Create the NeuronGroup** with `N_INHIBITORY` neurons, threshold `'v > V_THRESH_I'`, reset `'v = V_RESET_I*mV'`, refractory `REFRAC_I*ms`.

c **Initialise** `v = V_REST_I*mV`, `ge, gi = 0`.

TODO 3 — Input → Excitatory STDP Synapses

25 marks

These are the *plastic* synapses where all learning happens. STDP adjusts weights based on the relative timing of pre- and post-synaptic spikes.

STDP Rule (Triplet)

> 🧠 **SPIKE-TIMING-DEPENDENT PLASTICITY**
>
> Classical STDP: if a pre-synaptic spike arrives *before* a post-synaptic spike (causal), the synapse is strengthened (LTP). If post fires before pre (anti-causal), the synapse is weakened (LTD). Here a triplet rule is used: the LTP term involves a slow post-synaptic trace (`post2`), making weights sensitive to recent post-synaptic history.

Three exponentially-decaying traces are maintained (provided in the model string):

Trace	Resets to 1 when…	Time constant	Used for
`pre`	pre neuron fires	TAU_STDP_PRE (20 ms)	LTP
`post1`	post neuron fires	TAU_POST1 (20 ms)	LTD
`post2`	post neuron fires	TAU_POST2 (40 ms)	LTP (slow)

On Pre (pre neuron fires)

1. Deliver synaptic current: `ge_post += w`
2. Reset pre-trace: `pre = 1`
3. Apply LTD: `w = clip(w − LTD_RATE × post1, W_MIN, W_MAX)`

On Post (post neuron fires)

1. Snapshot slow trace: `post2before = post2`
2. Apply LTP: `w = clip(w + LTP_RATE × pre × post2before, W_MIN, W_MAX)`
3. Reset both post-traces: `post1 = 1` , `post2 = 1`

Steps to implement

a **Create** `input_group` : a `PoissonGroup` with `N_INPUT` neurons, rates starting at `0*Hz` .

b **Write** `stdp_on_pre` — three lines: conductance update, pre-trace reset, LTD weight update.

c **Write** `stdp_on_post` — four lines: snapshot post2, LTP update, reset post1, reset post2.

d **Create** `Synapses` connecting `input_group → exc_neurons` with the model and on_pre/on_post strings.

e **Connect all-to-all** with `stdp_synapses.connect()` and **initialise weights** using the formula above.

TODO 4 — Fixed Synapses: Excitatory ↔ Inhibitory

5 marks

Two fixed (non-plastic) synapse objects implement the winner-take-all circuit. These weights never change during training.

> **WHY I ≠ J?**
>
> When excitatory neuron *j* fires, it activates inhibitory neuron *j*. That inhibitory neuron then suppresses all excitatory neurons *except j itself* — the winner is not penalised, only the competitors are. This creates clean winner-take-all dynamics.

Section 4: Training TODO

The network assembly (Cell 10) and `normalize_weights()` helper are PROVIDED. Also, assigning the labels cell is PROVIDED (cell 12). **Your task is the training loop.**

TODO 5 — Training Loop

15 marks

Training Pipeline (per image)

1. **Weight normalisation** — rescale input→excitatory weights before each image (see below)
2. **Rate encoding** — convert pixel values to Poisson firing rates: `rates_hz = X_train[i] / 8.0 × input_intensity`
3. **Presentation** — set `input_group.rates = rates_hz * Hz`, run for `PRESENTATION_TIME` ms; STDP updates happen continuously
4. **Rest period** — set rates to `0*Hz`, run for `REST_TIME` ms; synaptic traces decay toward zero
5. **Spike check** — if total spikes < threshold, boost intensity and retry the same image

Steps to implement

a. Compute `rates_hz = X_train[i] / 8.0 * input_intensity`

b. Set `input_group.rates = rates_hz * Hz`

c. Run `net.run(PRESENTATION_TIME * ms)`

d. Record spikes: `current_spike_count = np.array(spike_mon.count) - previous_spike_count`

e. Set `input_group.rates = 0*Hz`, then run `net.run(REST_TIME * ms)`

Section 5: Testing TODO

TODO 6 — Test Phase

15 marks

After training, neuron labels are assigned (provided) and the network is evaluated on unseen test images. STDP must be disabled so learned weights are frozen.

How Classification Works

After training, each excitatory neuron is assigned to the digit class it responded to most (on average) during training. At test time:

1. Present test image → record which neurons fired and how many times
2. For each digit class, sum the spike counts of all neurons assigned to that class
3. Predict the class with the highest total activity

Steps to implement

a **Disable STDP** — set LTP and LTD rates to zero:

```
stdp_synapses.namespace['LTP_RATE'] = 0.0
stdp_synapses.namespace['LTD_RATE'] = 0.0
```

b Compute `rates_hz` from `X_test[i]` (same formula as training)

c Set `input_group.rates = rates_hz * Hz`

d Run `net.run(PRESENTATION_TIME * ms)`

e Use `predict_from_spikes(spike_counts_test, assignments)` and `compute_accuracy(y_test, predictions)`, then print the result

Section 6: Visualisation PROVIDED

Visualisation — Plots

5 marks

The plotting functions are provided — call them to produce the two plots below.

Spike Raster & Firing Rate Distribution (2 marks)

OBSERVE

Does your raster plot show *sparse* activity (only a few neurons firing per image) or *dense* activity (most neurons firing)? What does this tell you about whether winner-take-all inhibition is working?

Weight Receptive Fields (3 marks)

Each excitatory neuron has 784 incoming weights — one per input pixel. Reshaped to 28×28, these form a *receptive field*: what pattern does this neuron respond to most strongly?

OBSERVE

What you see in the receptive field plots:

- **If neurons show digit-like patterns:** Understand how STDP caused inputs that co-activate with a digit to have their weights strengthened while others were weakened.
- **If neurons show no clear structure (uniform colours):** Think what likely went wrong (e.g. too few training images, a bug in STDP, incorrect weight initialisation).

Either outcome is acceptable — you are graded on the quality of your interpretation, not on whether receptive fields look perfect.

Section 7: Written Analysis

TODO 7 — Written Analysis

15 marks

Answer each question in a **Markdown cell** in the notebook.

Q1 — ADAPTIVE THRESHOLD (3 MARKS)

What does the theta variable (θ) do during training? Explain its role as an intrinsic plasticity mechanism and how it prevents any single neuron from dominating the network.

Q2 — LTP/LTD ASYMMETRY (3 MARKS)

In `config.py`, `LTP_RATE = 0.01` and `LTD_RATE = 0.0001` — LTP is 100× stronger than LTD. Why is this asymmetry necessary? What would happen to the weights if `LTP_RATE = LTD_RATE = 0.005`?

Q3 — RECEPTIVE FIELD INTERPRETATION (3 MARKS)

Describe what you observe in your receptive field plots (Section 6). If you see digit-like structure, explain how STDP produced it. If you do *not* see clear structure, explain what went wrong and what specific change would produce better results.

Q4 — ACCURACY & IMPROVEMENT (3 MARKS)

State your test accuracy. Suggest **one specific** change to the network or training procedure (e.g. a parameter, architecture change, or longer training) and explain with biological reasoning why it would improve performance.

Q5 — STDP VS HEBBIAN LEARNING (3 MARKS)

In Week 7 you implemented Hebbian learning: weights increased when pre- and post-synaptic neurons were active at the same time. STDP also changes weights based on neural activity — but it is fundamentally different. What is the key difference, and why does that difference matter for learning *causal* relationships?